5.3.1 Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS (Manual)

Profile Applicability:

- Level 1

Description:

Encrypt Kubernetes secrets, stored in etcd, using secrets encryption feature during Amazon EKS cluster creation.

Rationale:

Kubernetes can store secrets that pods can access via a mounted volume. Today, Kubernetes secrets are stored with Base64 encoding, but encrypting is the recommended approach. Amazon EKS clusters version 1.13 and higher support the capability of encrypting your Kubernetes secrets using AWS Key Management Service (KMS) Customer Managed Keys (CMK). The only requirement is to enable the encryption provider support during EKS cluster creation.

Use AWS Key Management Service (KMS) keys to provide envelope encryption of Kubernetes secrets stored in Amazon EKS. Implementing envelope encryption is considered a security best practice for applications that store sensitive data and is part of a defense in depth security strategy.

Application-layer Secrets Encryption provides an additional layer of security for sensitive data, such as user defined Secrets and Secrets required for the operation of the cluster, such as service account keys, which are all stored in etcd.

Using this functionality, you can use a key, that you manage in AWS KMS, to encrypt data at the application layer. This protects against attackers in the event that they manage to gain access to etcd.

Audit:

Using the etcdctl commandline, read that secret out of etcd:

```
ETCDCTL_API=3 etcdctl get /registry/secrets/default/secret1 [...] | hexdump -
C
```

where [...] must be the additional arguments for connecting to the etcd server.
Verify the stored secret is prefixed with k8s:enc:aescbc:v1: which indicates the aescbc provider has encrypted the resulting data.

Remediation:

This process can only be performed during Cluster Creation.
Enable 'Secrets Encryption' during Amazon EKS cluster creation as described in the links within the 'References' section.

Default Value:

By default secrets created using the Kubernetes API are stored in *tmpfs* and are encrypted at rest.

References:

1. https://aws.amazon.com/about-aws/whats-new/2020/03/amazon-eks-adds-envelope-encryption-for-secrets-with-aws-kms/

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.11 Encrypt Sensitive Data at Rest Encrypt sensitive data at rest on servers, applications, and databases containing sensitive data. Storage-layer encryption, also known as server-side encryption, meets the minimum requirement of this Safeguard. Additional encryption methods may include application-layer encryption, also known as client-side encryption, where access to the data storage device(s) does not permit access to the plain-text data.		●	●
v7	14.8 Encrypt Sensitive Information at Rest Encrypt all sensitive information at rest using a tool that requires a secondary authentication mechanism not integrated into the operating system, in order to access the information.			●

5.4 Cluster Networking

This section contains recommendations relating to network security configurations in Amazon EKS.

5.4.1 Restrict Access to the Control Plane Endpoint (Manual)

Profile Applicability:

- Level 1

Description:

Enable Endpoint Private Access to restrict access to the cluster's control plane to only an allowlist of authorized IPs.

Rationale:

Authorized networks are a way of specifying a restricted range of IP addresses that are permitted to access your cluster's control plane. Kubernetes Engine uses both Transport Layer Security (TLS) and authentication to provide secure access to your cluster's control plane from the public internet. This provides you the flexibility to administer your cluster from anywhere; however, you might want to further restrict access to a set of IP addresses that you control. You can set this restriction by specifying an authorized network.

Restricting access to an authorized network can provide additional security benefits for your container cluster, including:

- Better protection from outsider attacks: Authorized networks provide an additional layer of security by limiting external access to a specific set of addresses you designate, such as those that originate from your premises. This helps protect access to your cluster in the case of a vulnerability in the cluster's authentication or authorization mechanism.
- Better protection from insider attacks: Authorized networks help protect your cluster from accidental leaks of master certificates from your company's premises. Leaked certificates used from outside Cloud Services and outside the authorized IP ranges (for example, from addresses outside your company) are still denied access.

Impact:

When implementing Endpoint Private Access, be careful to ensure all desired networks are on the allowlist (whitelist) to prevent inadvertently blocking external access to your cluster's control plane.

Audit:

Remediation:

By enabling private endpoint access to the Kubernetes API server, all communication between your nodes and the API server stays within your VPC. You can also limit the IP addresses that can access your API server from the internet, or completely disable internet access to the API server.
With this in mind, you can update your cluster accordingly using the AWS CLI to ensure that Private Endpoint Access is enabled.
If you choose to also enable Public Endpoint Access then you should also configure a list of allowable CIDR blocks, resulting in restricted access from the internet. If you specify no CIDR blocks, then the public API server endpoint is able to receive and process requests from all IP addresses by defaulting to ['0.0.0.0/0'].
For example, the following command would enable private access to the Kubernetes API as well as limited public access over the internet from a single IP address (noting the /32 CIDR suffix):

```
aws eks update-cluster-config --region $AWS_REGION --name $CLUSTER_NAME --
resources-vpc-config endpointPrivateAccess=true, endpointPrivateAccess=true,
publicAccessCidrs="203.0.113.5/32"
```

Note:
The CIDR blocks specified cannot include reserved addresses.
There is a maximum number of CIDR blocks that you can specify. For more information, see the EKS Service Quotas link in the references section.
For more detailed information, see the EKS Cluster Endpoint documentation link in the references section.

Default Value:

By default, Endpoint Public Access is disabled.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cluster-endpoint.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v8	9.3 Maintain and Enforce Network-Based URL Filters Enforce and update network-based URL filters to limit an enterprise asset from connecting to potentially malicious or unapproved websites. Example implementations include category-based filtering, reputation-based filtering, or through the use of block lists. Enforce filters for all enterprise assets.		●	●

Controls Version	Control	IG 1	IG 2	IG 3
v7	7.4 Maintain and Enforce Network-Based URL Filters Enforce network-based URL filters that limit a system's ability to connect to websites not approved by the organization. This filtering shall be enforced for each of the organization's systems, whether they are physically at an organization's facilities or not.		●	●

5.4.2 Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled (Manual)

Profile Applicability:

- Level 2

Description:

Disable access to the Kubernetes API from outside the node network if it is not required.

Rationale:

In a private cluster, the master node has two endpoints, a private and public endpoint. The private endpoint is the internal IP address of the master, behind an internal load balancer in the master's VPC network. Nodes communicate with the master using the private endpoint. The public endpoint enables the Kubernetes API to be accessed from outside the master's VPC network.

Although Kubernetes API requires an authorized token to perform sensitive actions, a vulnerability could potentially expose the Kubernetes publically with unrestricted access. Additionally, an attacker may be able to identify the current cluster and Kubernetes API version and determine whether it is vulnerable to an attack. Unless required, disabling public endpoint will help prevent such threats, and require the attacker to be on the master's VPC network to perform any attack on the Kubernetes API.

Impact:

Configure the EKS cluster endpoint to be private.

1. Leave the cluster endpoint public and specify which CIDR blocks can communicate with the cluster endpoint. The blocks are effectively a whitelisted set of public IP addresses that are allowed to access the cluster endpoint.
2. Configure public access with a set of whitelisted CIDR blocks and set private endpoint access to enabled. This will allow public access from a specific range of public IPs while forcing all network traffic between the kubelets (workers) and the Kubernetes API through the cross-account ENIs that get provisioned into the cluster VPC when the control plane is provisioned.

Audit:

Check for private endpoint access to the Kubernetes API server

Remediation:

By enabling private endpoint access to the Kubernetes API server, all communication between your nodes and the API server stays within your VPC.
With this in mind, you can update your cluster accordingly using the AWS CLI to ensure that Private Endpoint Access is enabled.
For example, the following command would enable private access to the Kubernetes API and ensure that no public access is permitted:

```
aws eks update-cluster-config --region $AWS_REGION --name $CLUSTER_NAME --
resources-vpc-config endpointPrivateAccess=true, endpointPublicAccess=false
```

Note: For more detailed information, see the EKS Cluster Endpoint documentation link in the references section.

Default Value:

By default, the Public Endpoint is disabled.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/cluster-endpoint.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	12 Boundary Defense Boundary Defense			

5.4.3 Ensure clusters are created with Private Nodes (Manual)

Profile Applicability:

- Level 1

Description:

Disable public IP addresses for cluster nodes, so that they only have private IP addresses. Private Nodes are nodes with no public IP addresses.

Rationale:

Disabling public IP addresses on cluster nodes restricts access to only internal networks, forcing attackers to obtain local network access before attempting to compromise the underlying Kubernetes hosts.

Impact:

To enable Private Nodes, the cluster has to also be configured with a private master IP range and IP Aliasing enabled.

Private Nodes do not have outbound access to the public internet. If you want to provide outbound Internet access for your private nodes, you can use Cloud NAT or you can manage your own NAT gateway.

Audit:

Remediation:

```
aws eks update-cluster-config \
    --region region-code \
    --name my-cluster \
    --resources-vpc-config
endpointPublicAccess=true,publicAccessCidrs="203.0.113.5/32",endpointPrivateA
ccess=true
```

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	4.4 Implement and Manage a Firewall on Servers Implement and manage a firewall on servers, where supported. Example implementations include a virtual firewall, operating system firewall, or a third-party firewall agent.	●	●	●
v7	12 Boundary Defense Boundary Defense			

5.4.4 Ensure Network Policy is Enabled and set as appropriate (Manual)

Profile Applicability:

- Level 1

Description:

Amazon EKS provides two ways to implement network policy. You choose a network policy option when you create an EKS cluster. The policy option can't be changed after the cluster is created: Calico Network Policies, an open-source network and network security solution founded by Tigera. Both implementations use Linux IPTables to enforce the specified policies. Policies are translated into sets of allowed and disallowed IP pairs. These pairs are then programmed as IPTable filter rules.

Rationale:

By default, all pod to pod traffic within a cluster is allowed. Network Policy creates a pod-level firewall that can be used to restrict traffic between sources. Pod traffic is restricted by having a Network Policy that selects it (through the use of labels). Once there is any Network Policy in a namespace selecting a particular pod, that pod will reject any connections that are not allowed by any Network Policy. Other pods in the namespace that are not selected by any Network Policy will continue to accept all traffic.

Network Policies are managed via the Kubernetes Network Policy API and enforced by a network plugin, simply creating the resource without a compatible network plugin to implement it will have no effect.

Impact:

Network Policy requires the Network Policy add-on. This add-on is included automatically when a cluster with Network Policy is created, but for an existing cluster, needs to be added prior to enabling Network Policy.

Enabling/Disabling Network Policy causes a rolling update of all cluster nodes, similar to performing a cluster upgrade. This operation is long-running and will block other operations on the cluster (including delete) until it has run to completion.

Enabling Network Policy enforcement consumes additional resources in nodes. Specifically, it increases the memory footprint of the kube-system process by approximately 128MB, and requires approximately 300 millicores of CPU.

Audit:

Remediation:

Default Value:

By default, Network Policy is disabled.

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	12.6 Use of Secure Network Management and Communication Protocols Use secure network management and communication protocols (e.g., 802.1X, Wi-Fi Protected Access 2 (WPA2) Enterprise or greater).		●	●
v7	9.2 Ensure Only Approved Ports, Protocols and Services Are Running Ensure that only network ports, protocols, and services listening on a system with validated business needs, are running on each system.		●	●
v7	9.4 Apply Host-based Firewalls or Port Filtering Apply host-based firewalls or port filtering tools on end systems, with a default-deny rule that drops all traffic except those services and ports that are explicitly allowed.	●	●	●

5.4.5 Encrypt traffic to HTTPS load balancers with TLS certificates (Manual)

Profile Applicability:

- Level 2

Description:

Encrypt traffic to HTTPS load balancers using TLS certificates.

Rationale:

Encrypting traffic between users and your Kubernetes workload is fundamental to protecting data sent over the web.

Audit:

Remediation:

References:

1. https://docs.aws.amazon.com/elasticloadbalancing/latest/userguide/data-protection.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	3.10 Encrypt Sensitive Data in Transit Encrypt sensitive data in transit. Example implementations can include: Transport Layer Security (TLS) and Open Secure Shell (OpenSSH).		●	●
v7	14.4 Encrypt All Sensitive Information in Transit Encrypt all sensitive information in transit.		●	●

5.5 Authentication and Authorization

This section contains recommendations relating to authentication and authorization in Amazon EKS.

5.5.1 Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes (Manual)

Profile Applicability:

- Level 2

Description:

Amazon EKS uses IAM to provide authentication to your Kubernetes cluster through the AWS IAM Authenticator for Kubernetes. You can configure the stock kubectl client to work with Amazon EKS by installing the AWS IAM Authenticator for Kubernetes and modifying your kubectl configuration file to use it for authentication.

Rationale:

On- and off-boarding users is often difficult to automate and prone to error. Using a single source of truth for user permissions reduces the number of locations that an individual must be off-boarded from, and prevents users gaining unique permissions sets that increase the cost of audit.

Impact:

Users must now be assigned to the IAM group created to use this namespace and deploy applications. If they are not they will not be able to access the namespace or deploy.

Audit:

To Audit access to the namespace $NAMESPACE, assume the IAM role yourIAMRoleName for a user that you created, and then run the following command:

```
$ kubectl get role -n $NAMESPACE
```

The response lists the RBAC role that has access to this Namespace.

Remediation:

Refer to the 'Managing users or IAM roles for your cluster' in Amazon EKS documentation.
Note: If using AWS CLI version 1.16.156 or later there is no need to install the AWS IAM Authenticator anymore.
The relevant AWS CLI commands, depending on the use case, are:

```
aws eks update-kubeconfig
aws eks get-token
```

Default Value:

For role-based access control (RBAC), system:masters permissions are configured in the Amazon EKS control plane

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/add-user-role.html
2. https://docs.aws.amazon.com/eks/latest/userguide/add-user-role.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	6.8 Define and Maintain Role-Based Access Control Define and maintain role-based access control, through determining and documenting the access rights necessary for each role within the enterprise to successfully carry out its assigned duties. Perform access control reviews of enterprise assets to validate that all privileges are authorized, on a recurring schedule at a minimum annually, or more frequently.			●
v7	16.2 Configure Centralized Point of Authentication Configure access for all accounts through as few centralized points of authentication as possible, including network, security, and cloud systems.		●	●

5.6 Other Cluster Configurations

This section contains recommendations relating to any remaining security-related cluster configurations in Amazon EKS.

5.6.1 Consider Fargate for running untrusted workloads (Manual)

Profile Applicability:

- Level 1

Description:

It is Best Practice to restrict or fence untrusted workloads when running in a multi-tenant environment.

Rationale:

Audit:

Remediation:

Create a Fargate profile for your cluster
Before you can schedule pods running on Fargate in your cluster, you must define a Fargate profile that specifies which pods should use Fargate when they are launched. For more information, see AWS Fargate profile.
Note
If you created your cluster with eksctl using the --fargate option, then a Fargate profile has already been created for your cluster with selectors for all pods in the kube-system and default namespaces. Use the following procedure to create Fargate profiles for any other namespaces you would like to use with Fargate.
via eksctl CLI
Create your Fargate profile with the following eksctl command, replacing the variable text with your own values. You must specify a namespace, but the labels option is not required.

```
eksctl create fargateprofile --cluster cluster_name --name
fargate_profile_name --namespace kubernetes_namespace --labels key=value
```

via AWS Management Console
To create a Fargate profile for a cluster with the AWS Management Console

1. Open the Amazon EKS console at https://console.aws.amazon.com/eks/home#/clusters.
2. Choose the cluster to create a Fargate profile for.
3. Under Fargate profiles, choose Add Fargate profile.
4. On the Configure Fargate profile page, enter the following information and choose Next.

- For Name, enter a unique name for your Fargate profile.
- For Pod execution role, choose the pod execution role to use with your Fargate profile. Only IAM roles with the eks-fargate-pods.amazonaws.com service principal are shown. If you do not see any roles listed here, you must create one. For more information, see Pod execution role.

- For Subnets, choose the subnets to use for your pods. By default, all subnets in your cluster's VPC are selected. Only private subnets are supported for pods running on Fargate; you must deselect any public subnets.
- For Tags, you can optionally tag your Fargate profile. These tags do not propagate to other resources associated with the profile, such as its pods.

5. On the Configure pods selection page, enter the following information and choose Next.

- list text hereFor Namespace, enter a namespace to match for pods, such as kube-system or default.
- Add Kubernetes labels to the selector that pods in the specified namespace must have to match the selector. For example, you could add the label infrastructure: fargate to the selector so that only pods in the specified namespace that also have the infrastructure: fargate Kubernetes label match the selector.

6. On the Review and create page, review the information for your Fargate profile and choose Create.

Default Value:

By default, AWS Fargate is not utilized.

References:

1. https://docs.aws.amazon.com/eks/latest/userguide/fargate.html

CIS Controls:

Controls Version	Control	IG 1	IG 2	IG 3
v8	16.8 Separate Production and Non-Production Systems Maintain separate environments for production and non-production systems.		●	●
v7	18.9 Separate Production and Non-Production Systems Maintain separate environments for production and nonproduction systems. Developers should not have unmonitored access to production environments.		●	●

Appendix: Summary Table

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
1	**Control Plane Components**		
2	**Control Plane Configuration**		
2.1	**Logging**		
2.1.1	Enable audit Logs (Manual)	☐	☐
3	**Worker Nodes**		
3.1	**Worker Node Configuration Files**		
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive (Manual)	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root (Manual)	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive (Manual)	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root (Manual)	☐	☐
3.2	**Kubelet**		
3.2.1	Ensure that the Anonymous Auth is Not Enabled (Automated)	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow (Automated)	☐	☐
3.2.3	Ensure that a Client CA File is Configured (Manual)	☐	☐
3.2.4	Ensure that the --read-only-port is disabled (Manual)	☐	☐
3.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0 (Automated)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
3.2.6	Ensure that the --protect-kernel-defaults argument is set to true (Automated)	☐	☐
3.2.7	Ensure that the --make-iptables-util-chains argument is set to true (Automated)	☐	☐
3.2.8	Ensure that the --hostname-override argument is not set (Manual)	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture (Automated)	☐	☐
3.2.10	Ensure that the --rotate-certificates argument is not present or is set to true (Manual)	☐	☐
3.2.11	Ensure that the RotateKubeletServerCertificate argument is set to true (Automated)	☐	☐
3.3	**Container Optimized OS**		
3.3.1	Prefer using a container-optimized OS when possible (Manual)	☐	☐
4	**Policies**		
4.1	**RBAC and Service Accounts**		
4.1.1	Ensure that the cluster-admin role is only used where required (Manual)	☐	☐
4.1.2	Minimize access to secrets (Manual)	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles (Manual)	☐	☐
4.1.4	Minimize access to create pods (Manual)	☐	☐
4.1.5	Ensure that default service accounts are not actively used. (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.1.6	Ensure that Service Account Tokens are only mounted where necessary (Manual)	☐	☐
4.1.7	Avoid use of system:masters group (Manual)	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster (Manual)	☐	☐
4.2	**Pod Security Policies**		
4.2.1	Minimize the admission of privileged containers (Automated)	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace (Automated)	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace (Automated)	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace (Automated)	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation (Automated)	☐	☐
4.2.6	Minimize the admission of root containers (Automated)	☐	☐
4.2.7	Minimize the admission of containers with added capabilities (Manual)	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned (Automated)	☐	☐
4.3	**CNI Plugin**		
4.3.1	Ensure CNI plugin supports network policies. (Manual)	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined (Manual)	☐	☐
4.4	**Secrets Management**		

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
4.4.1	Prefer using secrets as files over secrets as environment variables (Manual)	☐	☐
4.4.2	Consider external secret storage (Manual)	☐	☐
4.5	**Extensible Admission Control**		
4.6	**General Policies**		
4.6.1	Create administrative boundaries between resources using namespaces (Manual)	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers (Manual)	☐	☐
4.6.3	The default namespace should not be used (Manual)	☐	☐
5	**Managed services**		
5.1	**Image Registry and Image Scanning**		
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider (Manual)	☐	☐
5.1.2	Minimize user access to Amazon ECR (Manual)	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR (Manual)	☐	☐
5.1.4	Minimize Container Registries to only those approved (Manual)	☐	☐
5.2	**Identity and Access Management (IAM)**		
5.2.1	Prefer using dedicated EKS Service Accounts (Manual)	☐	☐
5.3	**AWS EKS Key Management Service**		
5.3.1	Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS (Manual)	☐	☐

CIS Benchmark Recommendation		Set Correctly	
		Yes	No
5.4	**Cluster Networking**		
5.4.1	Restrict Access to the Control Plane Endpoint (Manual)	☐	☐
5.4.2	Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled (Manual)	☐	☐
5.4.3	Ensure clusters are created with Private Nodes (Manual)	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate (Manual)	☐	☐
5.4.5	Encrypt traffic to HTTPS load balancers with TLS certificates (Manual)	☐	☐
5.5	**Authentication and Authorization**		
5.5.1	Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes (Manual)	☐	☐
5.6	**Other Cluster Configurations**		
5.6.1	Consider Fargate for running untrusted workloads (Manual)	☐	☐

Appendix: CIS Controls v7 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.6.1	Create administrative boundaries between resources using namespaces	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers	☐	☐
4.6.3	The default namespace should not be used	☐	☐
5.1.2	Minimize user access to Amazon ECR	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐

Appendix: CIS Controls v7 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.3	Ensure that a Client CA File is Configured	☐	☐
3.2.4	Ensure that the --read-only-port is disabled	☐	☐
3.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0	☐	☐
3.2.6	Ensure that the --protect-kernel-defaults argument is set to true	☐	☐
3.2.7	Ensure that the --make-iptables-util-chains argument is set to true	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
3.2.10	Ensure that the --rotate-certificates argument is not present or is set to true	☐	☐
3.2.11	Ensure that the RotateKubeletServerCertificate argument is set to true	☐	☐
3.3.1	Prefer using a container-optimized OS when possible	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Minimize access to secrets	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.3.1	Ensure CNI plugin supports network policies.	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined	☐	☐
4.4.1	Prefer using secrets as files over secrets as environment variables	☐	☐
4.6.1	Create administrative boundaries between resources using namespaces	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers	☐	☐
4.6.3	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Minimize user access to Amazon ECR	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐
5.1.4	Minimize Container Registries to only those approved	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.4.1	Restrict Access to the Control Plane Endpoint	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Encrypt traffic to HTTPS load balancers with TLS certificates	☐	☐
5.5.1	Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes	☐	☐
5.6.1	Consider Fargate for running untrusted workloads	☐	☐

Appendix: CIS Controls v7 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.3	Ensure that a Client CA File is Configured	☐	☐
3.2.4	Ensure that the --read-only-port is disabled	☐	☐
3.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0	☐	☐
3.2.6	Ensure that the --protect-kernel-defaults argument is set to true	☐	☐
3.2.7	Ensure that the --make-iptables-util-chains argument is set to true	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
3.2.10	Ensure that the --rotate-certificates argument is not present or is set to true	☐	☐
3.2.11	Ensure that the RotateKubeletServerCertificate argument is set to true	☐	☐
3.3.1	Prefer using a container-optimized OS when possible	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Minimize access to secrets	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.1.7	Avoid use of system:masters group	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.3.1	Ensure CNI plugin supports network policies.	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined	☐	☐
4.4.1	Prefer using secrets as files over secrets as environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.6.1	Create administrative boundaries between resources using namespaces	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers	☐	☐
4.6.3	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.1.2	Minimize user access to Amazon ECR	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐
5.1.4	Minimize Container Registries to only those approved	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.3.1	Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS	☐	☐
5.4.1	Restrict Access to the Control Plane Endpoint	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Encrypt traffic to HTTPS load balancers with TLS certificates	☐	☐
5.5.1	Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes	☐	☐
5.6.1	Consider Fargate for running untrusted workloads	☐	☐

Appendix: CIS Controls v7 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
	No unmapped recommendations to CIS Controls v7.0	☐	☐

Appendix: CIS Controls v8 IG 1 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐
4.1.2	Minimize access to secrets	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.3.1	Ensure CNI plugin supports network policies.	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined	☐	☐
5.1.2	Minimize user access to Amazon ECR	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐
5.4.1	Restrict Access to the Control Plane Endpoint	☐	☐
5.4.2	Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐

Appendix: CIS Controls v8 IG 2 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.3	Ensure that a Client CA File is Configured	☐	☐
3.2.4	Ensure that the --read-only-port is disabled	☐	☐
3.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0	☐	☐
3.2.6	Ensure that the --protect-kernel-defaults argument is set to true	☐	☐
3.2.7	Ensure that the --make-iptables-util-chains argument is set to true	☐	☐
3.2.8	Ensure that the --hostname-override argument is not set	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
3.2.10	Ensure that the --rotate-certificates argument is not present or is set to true	☐	☐
3.2.11	Ensure that the RotateKubeletServerCertificate argument is set to true	☐	☐
3.3.1	Prefer using a container-optimized OS when possible	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.2	Minimize access to secrets	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.1.7	Avoid use of system:masters group	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.3.1	Ensure CNI plugin supports network policies.	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined	☐	☐
4.4.1	Prefer using secrets as files over secrets as environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers	☐	☐
4.6.3	The default namespace should not be used	☐	☐
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Minimize user access to Amazon ECR	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐
5.1.4	Minimize Container Registries to only those approved	☐	☐
5.3.1	Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS	☐	☐
5.4.1	Restrict Access to the Control Plane Endpoint	☐	☐
5.4.2	Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Encrypt traffic to HTTPS load balancers with TLS certificates	☐	☐
5.6.1	Consider Fargate for running untrusted workloads	☐	☐

Appendix: CIS Controls v8 IG 3 Mapped Recommendations

Recommendation		Set Correctly	
		Yes	No
2.1.1	Enable audit Logs	☐	☐
3.1.1	Ensure that the kubeconfig file permissions are set to 644 or more restrictive	☐	☐
3.1.2	Ensure that the kubelet kubeconfig file ownership is set to root:root	☐	☐
3.1.3	Ensure that the kubelet configuration file has permissions set to 644 or more restrictive	☐	☐
3.1.4	Ensure that the kubelet configuration file ownership is set to root:root	☐	☐
3.2.1	Ensure that the Anonymous Auth is Not Enabled	☐	☐
3.2.2	Ensure that the --authorization-mode argument is not set to AlwaysAllow	☐	☐
3.2.3	Ensure that a Client CA File is Configured	☐	☐
3.2.4	Ensure that the --read-only-port is disabled	☐	☐
3.2.5	Ensure that the --streaming-connection-idle-timeout argument is not set to 0	☐	☐
3.2.6	Ensure that the --protect-kernel-defaults argument is set to true	☐	☐
3.2.7	Ensure that the --make-iptables-util-chains argument is set to true	☐	☐
3.2.8	Ensure that the --hostname-override argument is not set	☐	☐
3.2.9	Ensure that the --eventRecordQPS argument is set to 0 or a level which ensures appropriate event capture	☐	☐
3.2.10	Ensure that the --rotate-certificates argument is not present or is set to true	☐	☐
3.2.11	Ensure that the RotateKubeletServerCertificate argument is set to true	☐	☐
3.3.1	Prefer using a container-optimized OS when possible	☐	☐
4.1.1	Ensure that the cluster-admin role is only used where required	☐	☐

Recommendation		Set Correctly	
		Yes	No
4.1.2	Minimize access to secrets	☐	☐
4.1.3	Minimize wildcard use in Roles and ClusterRoles	☐	☐
4.1.4	Minimize access to create pods	☐	☐
4.1.5	Ensure that default service accounts are not actively used.	☐	☐
4.1.6	Ensure that Service Account Tokens are only mounted where necessary	☐	☐
4.1.7	Avoid use of system:masters group	☐	☐
4.1.8	Limit use of the Bind, Impersonate and Escalate permissions in the Kubernetes cluster	☐	☐
4.2.1	Minimize the admission of privileged containers	☐	☐
4.2.2	Minimize the admission of containers wishing to share the host process ID namespace	☐	☐
4.2.3	Minimize the admission of containers wishing to share the host IPC namespace	☐	☐
4.2.4	Minimize the admission of containers wishing to share the host network namespace	☐	☐
4.2.5	Minimize the admission of containers with allowPrivilegeEscalation	☐	☐
4.2.6	Minimize the admission of root containers	☐	☐
4.2.7	Minimize the admission of containers with added capabilities	☐	☐
4.2.8	Minimize the admission of containers with capabilities assigned	☐	☐
4.3.1	Ensure CNI plugin supports network policies.	☐	☐
4.3.2	Ensure that all Namespaces have Network Policies defined	☐	☐
4.4.1	Prefer using secrets as files over secrets as environment variables	☐	☐
4.4.2	Consider external secret storage	☐	☐
4.6.1	Create administrative boundaries between resources using namespaces	☐	☐
4.6.2	Apply Security Context to Your Pods and Containers	☐	☐
4.6.3	The default namespace should not be used	☐	☐

Recommendation		Set Correctly	
		Yes	No
5.1.1	Ensure Image Vulnerability Scanning using Amazon ECR image scanning or a third party provider	☐	☐
5.1.2	Minimize user access to Amazon ECR	☐	☐
5.1.3	Minimize cluster access to read-only for Amazon ECR	☐	☐
5.1.4	Minimize Container Registries to only those approved	☐	☐
5.2.1	Prefer using dedicated EKS Service Accounts	☐	☐
5.3.1	Ensure Kubernetes Secrets are encrypted using Customer Master Keys (CMKs) managed in AWS KMS	☐	☐
5.4.1	Restrict Access to the Control Plane Endpoint	☐	☐
5.4.2	Ensure clusters are created with Private Endpoint Enabled and Public Access Disabled	☐	☐
5.4.3	Ensure clusters are created with Private Nodes	☐	☐
5.4.4	Ensure Network Policy is Enabled and set as appropriate	☐	☐
5.4.5	Encrypt traffic to HTTPS load balancers with TLS certificates	☐	☐
5.5.1	Manage Kubernetes RBAC users with AWS IAM Authenticator for Kubernetes	☐	☐
5.6.1	Consider Fargate for running untrusted workloads	☐	☐

Appendix: CIS Controls v8 Unmapped Recommendations

Recommendation		Set Correctly	
		Yes	No
	No unmapped recommendations to CIS Controls v8.0	☐	☐

Appendix: Change History

Date	Version	Changes for this version
11/14/2022	1.2.0	Ticket #15928 – Recommendation 3.2.9 updated
11/14/2022	1.2.0	Ticket #15915 – Pod Security Policy deprecated
11/12/2022	1.2.0	Ticket #15913 – recommendation 3.2.9 updated Event QPS deprecated
11/12/2022	1.2.0	Ticket # 16516 – recommendation 3.2.2 updated configuration guidance regarding “always allow”
11/12/2022	1.2.0	Ticket #15485 – ELS Audit Log impact statement updated.
11/12/2022	1.2.0	Ticket #15654 – recommendation 3.2.3 audit
11/01/2022	1.2.0	Ticket #15618 – recommendation 3.2.9 profile updated to level 2
11/01/2022	1.2.0	Ticket #15616 – recommendation 3.2.8 - hostname-overide remdiation process updated.
11/01/2022	1.2.0	Ticket #15805 – recommendation 3.3.1 audit procedure updated.

Date	Version	Changes for this version
10/10/2022	1.2.0	Ticket #15843 – remediation process updated to set *.spec.privileged field to false.*
10/10/2022	1.2.0	Ticket #15849 – PSP has allowedCapabilities Set by Default
10/10/2022	1.2.0	Ticket #15850 – recommendation 4.2.7 combined with 4.2.9
10/08/2022	1.2.0	Ticket #15532 – recommendation 5.4.2 updated remediation process
10/08/2022	1.2.0	Ticket # 15804 – recommendation 3.3.1 description updated
10/08/2022	1.2.0	Ticket #15602 – recommendation 3.2.6 audit procedure updated
10/08/2022	1.2.0	Ticket # 16433 – recommendation 3.2.10 updated the default value
10/08/2022	1.2.0	Ticket #15613 – recommendations 3.2.1, 3.2.2, 3.2.3 updated to reflect authentication vs authorization exec arguments
10/08/2022	1.2.0	Ticket #15677 – recommendation 5.2.1 updated to reflect EKS specifics

Date	Version	Changes for this version
10/08/2022	1.2.0	Ticket #15533 recommendation 5.4.4 updated recommendation for EKS specifics
10/08/2022	1.2.0	Ticket #
9/01/2022	1.2.0	Ticket # 15615 – recommendation 3.2.8 updated impact statement
9/01/2022	1.2.0	Ticket # 15611 – recommendation 3.2.5 updated audit and remediation methods
9/01/2022	1.2.0	Ticket # 15610 – recommendation 3.2.6 updated description
9/01/2022	1.2.0	Ticket # 15531 – recommendation 5.4.1 updated remediation process
9/01/2022	1.2.0	Ticket #15533 – recommendation 5.4.4 updated
9/01/2022	1.2.0	Ticket #15599 – recommendation 5.4.5 reviewed load balancer scope
9/01/2022	1.2.0	Ticket #15554 – recommendation 5.5.1 updated default configuration
9/01/2022	1.2.0	Ticket #16573 - recommendation 3.2.1 and 3.2.2 updated and aligned based on new functionality

Date	Version	Changes for this version
9/01/2022	1.2.0	Ticket #16612 – recommendation 5.4.2 updated audit procedure
9/01/2022	1.2.0	Ticket #16674 – recommendation 3.2.3 updated out of date guidance
9/01/2022	1.2.0	Ticket #15533 – recommendation 5.5.1 updated description
9/01/2022	1.2.0	Ticket #15539 – recommendation 5.4.3 updated remediation process
9/01/2022	1.2.0	Ticket #15614 – recommendation 3.2.1 updated remediation process
9/01/2022	1.2.0	Ticket #17054 – recommendation 5.4.1 updated for EKS specifically